



VectorLiteRAG: Latency-Aware and Fine-Grained Resource Partitioning for Efficient RAG

Junkyum Kim and Divya Mahajan
Georgia Institute of Technology
{jun-kyum.kim, divya.mahajan}@gatech.edu

Abstract—Retrieval-Augmented Generation leverages vector similarity search to enhance large language models with up-to-date, external knowledge, enabling accurate and reliable responses. While CPU-only vector search incurs high latency on large, high-dimensional indices, co-locating the retriever and the LLM on the GPU leads to resource sharing that can create resource contention. Specifically, vector search is memory and I/O intensive, placing it in direct conflict with LLM inference, which demands memory for KV cache and compute for higher throughput. We present VECTORLITERAG, a latency-aware RAG serving system that explicitly orchestrates data placement and execution across retrieval and inference to meet strict end-to-end SLOs. VECTORLITERAG is driven by access-pattern analysis and performance estimation to regulate how retrieval variability can be mitigated and managed in the system with LLM inference, enabling SLO-compliant execution under skewed and dynamic workloads. By jointly modeling search latency and query hit-rate distributions, VECTORLITERAG identifies an optimal index partitioning point across CPU and GPU that minimizes contention and stabilizes batching behavior, thereby maximizing sustained throughput under skewed access patterns. A low-overhead online index update mechanism allows VECTORLITERAG to continuously adapt to evolving request distributions, preserving batching efficiency and throughput as access patterns evolve. Our evaluations demonstrate that VECTORLITERAG consistently expands the range of SLO-compliant request rate across all tested configurations. Without increasing the generation latency or requiring additional hardware, VECTORLITERAG outperforms both naive and existing alternative frameworks, improving attainable SLO-bound throughput by up to 1.5 $\times$.

I. INTRODUCTION

Retrieval-Augmented Generation (RAG) is a powerful system in natural language processing, particularly for domain-specific question answering and information retrieval tasks [7], [8], [10], [22], [33]. Its key strength lies in combining parametric memory, encoded in the weights of a large language model, with non-parametric memory retrieved from an external knowledge corpus. Although parametric memory provides strong generalization, it is expensive to train and difficult to update. To mitigate this, RAG pipelines first perform similarity search using approximate nearest neighbor search (ANNS) algorithms to retrieve relevant documents from a large database. The retrieved documents are then fed into the LLM's context to generate up-to-date and reliable responses.

RAG frameworks [20], [24], [38] typically adopt heterogeneous hardware configurations, where vector retrieval is executed on CPUs and LLM generation is served by GPUs. This is driven by the system characteristics: LLM inference requires

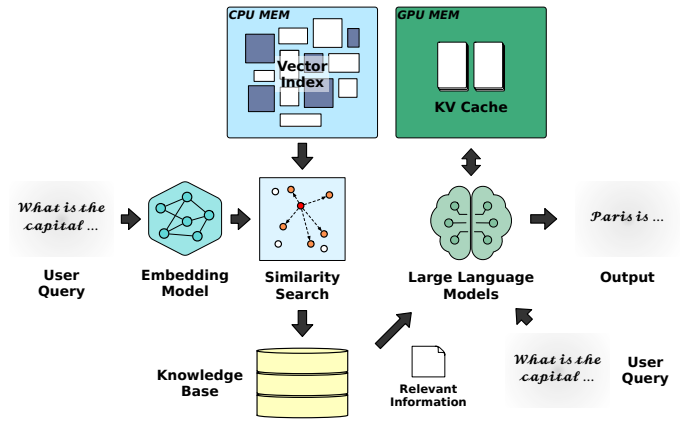


Fig. 1. End-to-end pipeline of a RAG system, where the input query is indexed into the vector database stored in memory, while the knowledge corpus resides in storage. The LLM prefill and decode execute on the GPU.

massive matrix multiplications and benefits significantly from GPU acceleration, whereas retrieval has traditionally been seen as a lighter task suited for CPUs. Offloading retrieval to CPUs allows GPUs to be dedicated to the more compute-intensive generation phase. CPU-based vector search may be sufficient for small vector databases, however, as the dimensionality of the embeddings and the size of the dataset grow, retrieval becomes increasingly compute- and memory- bound. CPUs, with limited parallelism, narrower vector units, and lower memory bandwidth, struggle to handle high-throughput similarity search at scale.

This latency imbalance creates a bottlenecked pipeline where the relatively slow CPU-based retrieval delays the GPU-accelerated generation phase, reducing the benefits of fast LLM inference and degrading overall system responsiveness. In our observations, CPU-based retrieval can take up to twice as long as the LLM prefill phase, increasing the total Time-to-First-Token (TTFT) from 197ms to 606ms when using a large database with 128M vectors, compared to a language model (Llama3-8B) operating without retrieval.

Although the retrieval operation is computationally lighter than the generation phase, it can still benefit significantly from GPU acceleration for two reasons: (1) GPUs feature wide and powerful vector units that enable highly parallelized distance computations, offering superior performance for similarity calculations on long embedding vectors. (2) The retrieval process involves scanning intermediate distance tensors to

identify the closest data points in the vector space. These operations are typically implemented as memory lookups a task where GPUs outperform CPUs due to their vectorized memory access and higher I/O.

In addition to compute and bandwidth demands, vector retrieval introduces significant memory pressure. To reduce memory footprint and speed up the search process, vector databases are commonly compressed into vector indexes using quantization techniques such as product quantization (PQ) [13]. Nevertheless, even after compression, vector indexes still occupy significant memory space, often exceeding the memory capacity of a GPU. Furthermore, intermediate data structures such as distances between cluster centroids and queries consume additional memory.

These compute and memory pressures create a resource tension between the retrieval and generation stages, especially as the vector database grows and CPU-based search fails to meet strict latency requirements. GPU memory is already constrained, with most of it reserved for model weights and KV cache for the LLM. Naively sharding the vector index across all GPUs can lead to memory contention and reduced overall throughput. Alternatively, assigning a disaggregated GPU for retrieval can prevent direct interference between stages, but degrades overall system throughput by reducing the number of available LLM instances, in particular when models require multiple GPUs, enforcing rigid allocation schemes.

Motivated by these challenges, this work explores a holistic approach to optimizing distributed RAG pipelines through joint resource allocation between vector search and LLM generation. We present VECTORLITERAG, a system that partitions the vector index between GPU and CPU-based on query access patterns and LLM deployment configurations, aiming to maximize throughput while meeting latency targets by exploiting the compute power of GPUs across both stages of the RAG pipeline. By analytically modeling similarity search latency, we determine the smallest index portion that needs to be placed on the GPU to satisfy the latency requirement under a given system configuration. Accordingly, VECTORLITERAG offers a latency-aware, throughput-optimized solution that requires no additional hardware resources. This approach is grounded in two key insights:

Access-Skew-Aware Data Layout. VECTORLITERAG leverages a key characteristic of Inverted File (IVF) based retrieval systems [46], that query accesses exhibit skew across clusters. To take advantage of this, VECTORLITERAG incorporates an analytical model that determines the optimal partitioning point and corresponding layout for a multi-GPU system. While the coarse quantizer and cold clusters remain on the CPU, a small subset of hot clusters are cached and distributed across GPUs. The system allocates just enough hot clusters to the GPUs, avoiding both oversubscription of GPU resources during retrieval.

Inter/Intra-Query Variance-Aware Routing. When hot clusters are distributed across GPUs, hit rates vary both across queries (inter-query variance) and across device shards within a query (intra-query variance). Existing systems that enforce

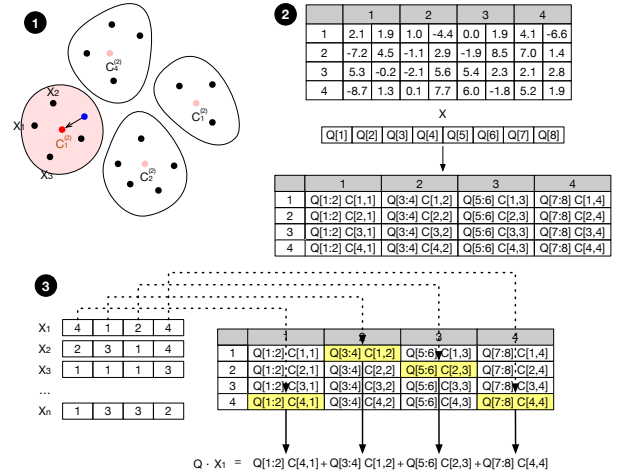


Fig. 2. Three stages of vector search in IVF-based index: (1) coarse quantization to identify clusters most semantically similar to the query, (2) construction of a LUT containing partial distances between the query and codewords, and (3) scanning the LUT and re-ranking candidates from the selected clusters based on aggregated distances.

fixed retrieval configurations across devices fail to account for this variability and often over-allocate GPU threads. VECTORLITERAG introduces query- and shard- aware routing to avoid such inefficiencies. After determining the most relevant clusters, it dispatches work to CPU or GPU based on their actual expected contribution. It also monitors per-query progress, forwarding early-finishing queries to reduce straggler-induced delays and improve batching efficiency.

Our contributions are summarized as follows:

- **Access-skew modeling and hit-rate estimation.** We characterize access skew in IVF-based retrieval systems and develop a hit-rate estimation method based on observed cluster access patterns.
- **Analytical latency model and SLO-aware partitioning.** We construct a latency model that accounts for inter-query variance and use it to determine the optimal CPU-GPU index partitioning point that meets latency targets.
- **Distributed retrieval pipeline.** We design a distributed retrieval pipeline that adaptively allocates search tasks across CPUs and GPUs by exploiting inter-device hit rate variance, improving efficiency and avoiding unnecessary GPU resource usage.

II. RETRIEVAL AUGMENTED GENERATION

In a RAG system, user queries are first transformed into vector embeddings using embedding models [30], [34], [36], [42]. These embeddings capture the semantics of the input and enable similarity search by comparing query vectors to a vector database constructed from the knowledge corpus, typically encoded using the same embedding model. State-of-the-art embedding models produce vectors of several thousand dimensions for higher quality, but this increased dimensionality raises the cost of distance computations.

Since exhaustive pairwise search is computationally infeasible at scale, large vector retrieval relies on approximate nearest

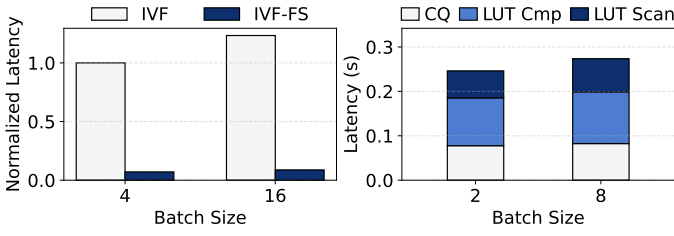


Fig. 3. **Left:** Search latency comparison between standard IVF and IVF with fast scan (IVF-FS). Except for the fast scan optimization, both indexes share identical configurations. IVF-FS achieves significantly faster search speed. **Right:** Latency breakdown of IVF-FS on a 128M vector index. Lookup table operations dominate the overall search time.

neighbor search to efficiently identify relevant documents. The retrieved vectors are mapped back to their corresponding documents, which are provided as additional context to the LLM alongside the original query.

A. Inverted List Index IVF

There are several approaches for structuring a vector database into a searchable index. Among them, HNSW and IVF are the most widely used.

HNSW [27] (Hierarchical Navigable Small World) is a graph-based structure where each vector forms a node connected to its nearest neighbors. It enables rapid search via hierarchical traversal and offers fast index construction. However, the additional edge information significantly increases memory usage as the dataset grows.

In contrast, the Inverted File (IVF) index [46] organizes the index as a hierarchical clustering structure. A subset of vectors is first clustered via K-Means to obtain centroids. Then, each database vector is assigned to the closest centroid, forming an inverted list. This structure narrows the search space using only centroid metadata, resulting in low memory overhead and high scalability. As such, IVF is widely adopted and studied in retrieval systems for large knowledge corpora [6], [11], [14], [21], [35], [43]. To further reduce memory usage, quantization techniques are applied on top of IVF. Scalar quantization (SQ) reduces each vector element to a smaller numerical type (e.g., float32 to int8), offering simplicity but limited compression. For higher compression ratios, product quantization (PQ) [13] is commonly used.

B. Search Operation in IVF Index

Figure 2 illustrates the search process in an IVF-PQ index, where an inverted list structure is combined with product quantization. When a query is received, the retriever first identifies the closest clusters, narrowing the search space. The number of clusters searched is controlled by the parameter *nprobe*, which trades off speed and accuracy.

Next, a distance lookup table is constructed. Since each vector is quantized into discrete sub-vector codes, each code maps to a representative value, trained and stored in the codebook. By pre-computing distances between the query vector and these representative values, the system avoids computing full distances to every vector. During the scan stage, these

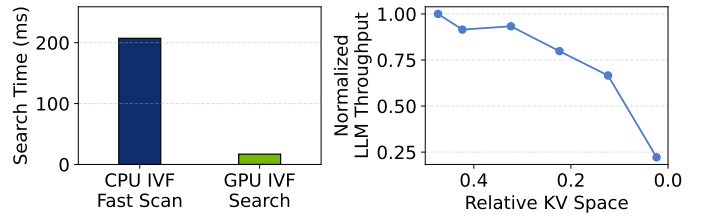


Fig. 4. **Left:** While fast scanning accelerates IVF-based vector search on CPU(64 core Xeon 8462Y+), GPU(H100)-based IVF search offers superior performance. **Right:** Relationship between KV cache size and LLM throughput for the Qwen3-30B model on two H100 GPUs. Reducing KV cache space leads to a significant drop in throughput.

LUTs are used to accumulate approximate distances and retrieve the top-k nearest vectors.

A deeper analysis of IVF search, shown in Figure 3, reveals that the large portion of the search time is spent on constructing and scanning the distance lookup table. This highlights the LUT stage as a key bottleneck in retrieval latency. To mitigate this overhead, fast scanning techniques [4] have been proposed and implemented in libraries such as Faiss [6] and ScaNN [43]. These methods leverage SIMD instructions and CPU vector registers to accelerate distance lookup operations. By carefully organizing lookup tables and quantization codes into memory-aligned layouts, they significantly outperform conventional IVF scan routines, particularly in CPU-based environments.

Motivated by their superior latency-performance trade-off, we adopt fast scanning in our system to enable efficient and low-latency vector retrieval. However, despite the SIMD capabilities of modern CPUs, CPU-based search can still become a bottleneck, ultimately degrading the responsiveness of the end-to-end RAG system.

III. CHALLENGES AND OPPORTUNITIES IN RAG SERVING

A. GPU search vs. CPU search

While fast scan indexes significantly improve the latency of vector similarity search on CPUs, GPU-based retrieval can offer even greater speedups, due to their wider vector processing units and higher memory bandwidth. As shown in Figure 4 (left), GPU-accelerated IVF search can outperform fast scan methods by nearly an order of magnitude.

Thus, offloading retrieval to the GPU can offer higher speedups for large-scale vector databases where CPU-based search remains a bottleneck. However, this comes with a fundamental trade-off: GPU memory is already heavily utilized by LLMs, particularly for storing KV cache and model weights. Allocating additional memory for the vector index can reduce available cache space, ultimately degrading LLM throughput, as illustrated in Figure 4 (right).

Beyond memory capacity, GPU retrieval additionally incurs scheduling overheads due to increased contention for compute resources. Shared memory is used to stage partial distance lookup tables, and each query-cluster pair typically maps to a thread block. As the number of probed clusters increases, so does the occupancy and scheduling pressure on the GPU, further impacting performance.

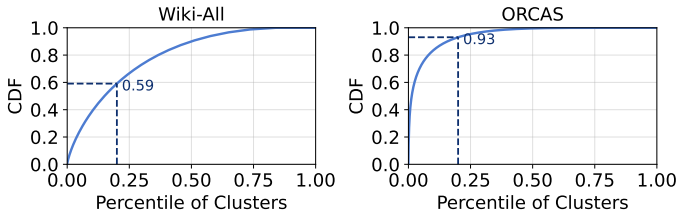


Fig. 5. CDF of cluster access frequency for queries from the Wiki-All [37] and ORCAS [5] datasets. While the two distributions exhibit different levels of skewness, in both cases, the top 20% of clusters account for over 50% of the total distance computations.

Takeaway 1. *GPU-based retrieval can substantially outperform even the fastest CPU-based methods, but due to contention with LLM inference workloads, careful memory and compute allocation is essential.*

B. Opportunity of Tiered Search Structure

The distribution of query access patterns in IVF indexes reveals the presence of hot clusters, a small subset that dominates retrieval traffic.

As shown on the left of Figure 5, the cumulative distribution of coarse quantization results exhibits a strong skew: the top 20% of clusters account for nearly 60% of accesses in Wiki-All [37] and over 93% in ORCAS [5]. This skew is especially pronounced in ORCAS, which reflects real-world query behavior through unfiltered click-through logs, capturing both popularity bias and the imbalance introduced by k-means quantization.

This imbalance results in inefficient memory usage, as significant resources are allocated to rarely accessed clusters with limited contribution to retrieval quality.

Takeaway 2. *IVF index access patterns are highly skewed: a small number of clusters account for the vast majority of retrievals. This motivates a tiered index design, where frequently accessed clusters are prioritized for acceleration (e.g., GPU caching), and cold clusters are offloaded to lower-tier compute and storage.*

Embedding access patterns in recommendation systems are also known to exhibit significant skew, where a small subset of items or users dominates embedding lookup frequency. This observation has motivated several tiered architecture designs that prioritize popular embeddings for faster access [1]–[3], [19], [25], [25], [29]. Inspired by this insight, our work offers tiered acceleration to vector similarity search. However, a key distinction lies in the granularity of memory accesses. In recommendation systems, embedding look-ups are performed via embedding IDs. In contrast, vector similarity search systems conduct fully content-based retrieval, where relevant vectors must be located by computing distances to hundreds or thousands of candidates per query. To identify the nearest vector, the search must access not only the target vector but also neighboring vectors within the cluster.

Moreover, even if each embedding is uniformly accessed, clusters can contain varying numbers of vectors, exacerbating the access skew. This imbalance causes certain clusters to dominate query traffic, creating hot regions in memory access.

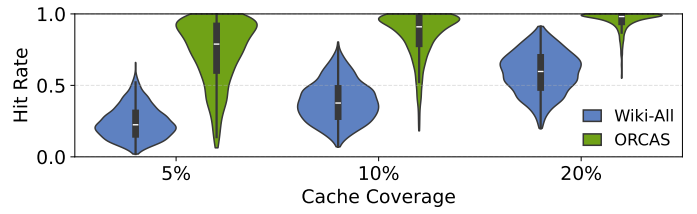


Fig. 6. Violin plot of hit rate distribution at different cache-coverages. The width of the violin indicates the density of queries with similar hit rates, while the white dot and black bar denote the median and inter-quartile range, respectively. This highlights that increasing cache coverage improves overall hit rates but does not eliminate tail queries with poor hit rates.

As a result, skew in our setting emerges more prominently at the cluster level rather than the vector level.

Consequently, although both domains benefit from tiered designs, the unit of optimization and the manifestation of skew differ substantially. Our approach explicitly targets cluster-level skew in large-scale retrieval workloads, enabling effective tiered placement and latency-aware resource allocation that are not directly addressed by prior embedding-centric designs.

C. Variance of Hit Rate across Queries

While tiered resource allocation strategies can accelerate vector search by caching frequently accessed clusters, their effectiveness in deployment is often hindered by query-level variance in hit rates. Long-tail queries with less cache hits can significantly limit the overall performance gains.

Figure 6 presents a violin plot of hit rate distributions across queries, measured by counting the number of clusters (among the total n_{probe}) that fall within the cached hot cluster set. As cache coverage increases from 5% to 20% of total clusters, the average hit rate improves accordingly. However, the variance remains substantial, especially in highly skewed datasets such as ORCAS, where a long tail of queries exhibits minimal cache benefit.

This variance introduces a deployment challenge. Since vector search throughput scales with batch size, retrievers are typically deployed with batching enabled. However, in the presence of low-hit queries within a batch, the entire batch’s processing time is effectively bounded by the slowest query. As a result, even if the average per-query latency is reduced by GPU acceleration, end-to-end latency improvements are constrained. Therefore, to fully realize the benefits of tiered or cached retrieval in real-world deployments, it is essential to account for such hit rate variance and long-tail behavior during system design.

Takeaway 3. *Variance in hit rate across queries poses a challenge in latency-critical deployments, due to long-tail queries as batching amplifies the impact of long-tail queries, limiting the effectiveness of caching.*

In summary, while GPU-based retrieval can vastly outperform CPU methods, it introduces a resource contention problem when co-located with LLMs, due to limited GPU memory and compute capacity. Meanwhile, query access patterns exhibit strong skew: a small fraction of clusters account for most retrieval traffic, making selective caching and tiered

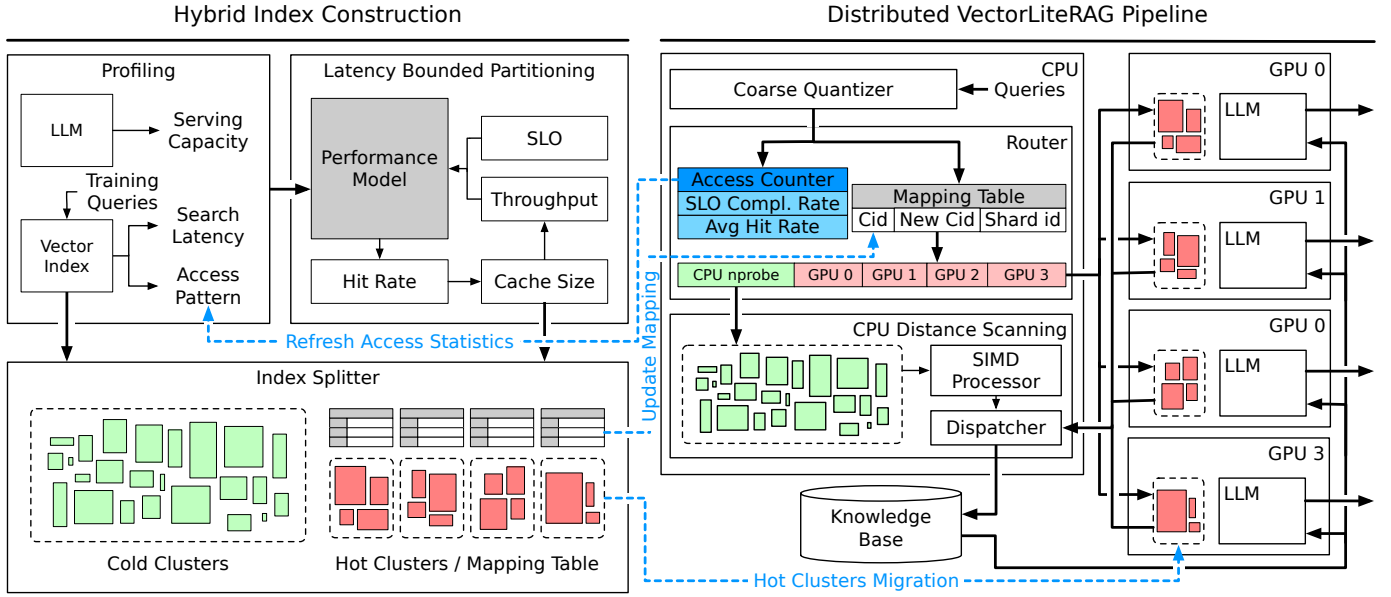


Fig. 7. System architecture of VECTORLITERAG. The system has two stages, **Left**: offline hybrid index construction and **Right**: runtime distributed pipeline. Profiling guides latency-bound partitioning to determine cache size and split point, producing sharded indices and mapping tables. At runtime, queries are routed via coarse quantizer and mapping tables, hot clusters run on GPUs, cold clusters on CPUs. A dynamic dispatcher forwards early-finished queries to LLM workers in a timely manner. Blue trails and boxes indicate runtime index refresh and update procedures.

search strategies effective. However, significant variance in hit rates across queries, especially long-tail queries, poses a major challenge in latency-sensitive deployments, as batching magnifies the bottleneck introduced by slow queries. These insights motivate the design of VECTORLITERAG, which adaptively partitions the index across GPU and CPU tiers, accounting for workload skew, hit rate variance, and end-to-end latency constraints to optimize throughput and responsiveness.

IV. VECTORLITERAG

VECTORLITERAG is an optimized RAG system that determines the optimal configuration for a CPU–GPU hybrid vector index. It is organized around tightly integrated components: (1) performance modeling and latency-aware hybrid index construction, and (2) a distributed runtime pipeline for inference serving. Given the latency constraint, LLM, index, and system configuration, VECTORLITERAG computes a partitioning point for tiered search, constructs the hybrid index, and serves inference requests through a tailored pipeline.

Hybrid Index Construction. The first component of VECTORLITERAG focuses on understanding the performance characteristics of the underlying system. This stage profiles CPU-based search latency, query-to-cluster access patterns, and standalone LLM throughput to characterize contention between retrieval and generation. These measurements drive a performance model and cache-coverage estimator, enabling a latency-bounded partitioning algorithm to select hot clusters. The hot clusters are then sharded into GPU sub-indices.

Distributed VECTORLITERAG Pipeline. The second component is the runtime pipeline that operationalizes the hybrid index. At runtime, batched queries are routed to CPU

or GPU shards using mapping tables generated during index construction, allowing each shard to operate with a flexible *nprobe* budget and reducing contention with LLM. A dynamic dispatcher further improves batching efficiency by advancing early-completing queries to mitigate tail latency.

The partitioning scheme and runtime pipeline are independent of the distance metric or compression method. As long as the index exhibits clustered structure and benefits from GPU acceleration, VECTORLITERAG can identify an effective hybrid configuration and deliver SLO-compliant RAG service.

A. Hybrid Index Construction

1) Profiling-based Performance Modeling: Since GPU resources are limited, accurately modeling performance is critical for determining the optimal index partitioning point. To construct these models, VECTORLITERAG profiles latency and access statistics using calibration queries from a training set. Specifically, it collects: (1) latency breakdown of CPU-based vector search and (2) cluster access frequency distributions. Additionally, throughput of the bare LLM is measured to guide partitioning decisions under joint CPU-GPU execution.

As described in Section II-B, IVF index search latency is dominated by two components: coarse quantization (CQ) and LUT operations. We profile both stages across varying batch sizes and construct independent models for each. However, in our design, only the LUT stage, which corresponds to the individual distance computation and scanning step, is considered for GPU offloading for two main reasons:

First, CQ is a similarity search over the quantizer (centroid) vectors, which is often implemented using memory-intensive graph-based structures such as HNSW. Offloading CQ to GPU

would require additional memory for the graph and complicate memory management. Second, if CQ were distributed across GPU shards, the resulting search path would involve repeated device transitions: CPU $\rightarrow$ GPU (quantization) $\rightarrow$ CPU (merge and routing) $\rightarrow$ GPU (search) $\rightarrow$ CPU (final merge). This induces costly inter-device communication and synchronization overheads. Moreover, our objective is to ensure stable performance within the latency budgets rather than to minimize absolute latency. Thus, for our purpose, we retain CQ on the CPU and use GPUs for distance computations, as this offers performance benefits while simplifying the optimization space.

Empirically, as shown in Figure 8 (left), CPU search latency exhibits a piecewise linear relationship with batch size. Initial steps appear as the system transitions from single-threaded (single query) to multi-threaded execution (batched queries). Accordingly, we model T_{CQ}^{CPU} and T_{LUT}^{CPU} as piecewise linear functions of batch size.

When hot clusters are cached, the overall search time reduces accordingly. LUT operations offloaded to GPUs are fully hidden under CPU’s execution, and the CPU processing time decreases in proportion to the number of hits. As a result, we model the latency of the hybrid partitioned index as:

$$\tau_s(b) = T_{CQ}^{CPU}(b) + (1 - \eta) \cdot T_{LUT}^{CPU}(b) \quad (1)$$

where η denotes the hit rate, in particular the minimum hit rate among all queries in the batch.

2) *Tail Query Hit Rate Estimation:* As discussed in Section III-C, caching hot clusters leads to varying hit rates across queries. Because, CPU side LUT workload is proportional to the miss rate $(1 - \eta)$, this variance directly translates into differences in search latency. Moreover, since vector search is typically executed in batches to maximize throughput, the completion time of the entire batch is dictated by the slowest query, one with the fewest hits. Therefore, modeling the minimum hit rate within a batch is critical for accurate performance estimation.

We model the distribution of per-query hit rates using a Beta distribution $f(x)$, which is widely used in Bayesian statistics for variables constrained to the $[0, 1]$ range. For a batch of size b , the expected minimum hit rate $\eta_{\min}$, i.e., the first-order statistic, is computed as:

$$\eta_{\min}(B) = \int_0^1 B \cdot x \cdot f(x) \cdot (1 - F(x))^{B-1} dx \quad (2)$$

where $F(x)$ is the cumulative distribution function of $f(x)$.

The mean hit rate $\bar{\eta}$ can be obtained directly from the query-cluster access profile, which reflects the cumulative fraction of accesses covered by the cached clusters. Estimating the variance is more challenging, as it would require re-running queries through the quantizer and counting individual hits after masking hot clusters, a process that is both computationally expensive and incompatible with iterative partitioning algorithm.

Instead, we approximate the hit rate variance as a function of the mean. We observe that hit/miss variance peaks when

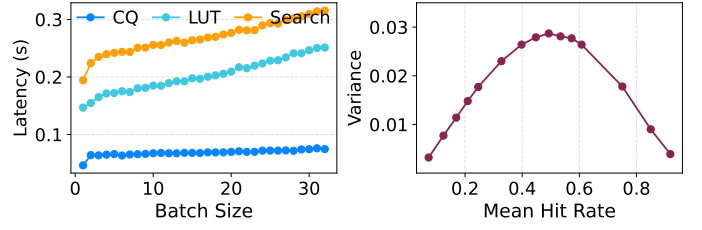


Fig. 8. **Left:** Search latency of ORCAS queries on a 64-core Intel Xeon 8426Y CPU. **Right:** Empirical variance of hit rates across queries in the Wiki-All dataset as a function of mean hit rate. The observed parabolic shape supports our variance approximation model.

$\bar{\eta} = 0.5$, and becomes more uniform as $\bar{\eta} \rightarrow 0$ or $\bar{\eta} \rightarrow 1$. This mirrors the variance behavior of the Beta distribution; $\text{Var}(X) \propto \bar{\eta}(1 - \bar{\eta})$. Thus, by empirically profiling the variance at $\bar{\eta} = 0.5$, denoted $\sigma_{\max}^2$, we can approximate the variance at arbitrary $\bar{\eta}$ as:

$$\sigma^2 \approx 4 \cdot \sigma_{\max}^2 \cdot \bar{\eta}(1 - \bar{\eta})$$

Figure 8 (right) validates the approximation. This allows instantiating a Beta distribution $f(x)$ with inferred mean and variance for any cache coverage configuration.

Finally, using Eq. 2, we compute the minimum hit rate within a batch for a given cache coverage. Inverting this relation numerically yields the function:

$$\rho = \text{HitRate2Converge}(B, \eta_{\min})$$

which is used in the main partitioning algorithm to identify the optimal cache coverage that satisfies latency constraints.

3) *Latency-Bounded Partitioning Algorithm:* In the hybrid RAG pipeline, LLM throughput decreases as more GPU memory is allocated to the vector index, due to contention between KV cache and index storage. To balance these competing demands, we introduce an iterative algorithm that determines an index partitioning point satisfying the latency constraint.

Algorithm 1 outlines the proposed latency-bounded partitioning algorithm. It takes the following inputs: the latency target, the baseline KV cache memory footprint when no vector index is loaded, and the peak bare LLM throughput. The goal is to find the largest feasible cache coverage for the GPU index (partitioning point ρ) that satisfies SLO constraint.

We first compute the latency bound for the hybrid vector search stage. To account for queuing delay, the analysis considers a worst-case scenario in which a request arrives immediately after the previous batch begins processing. Under steady-state load with uniformly arriving requests, this tail query experiences full batch latency $W(b)$ as queuing delay. To maintain the total response time within the latency budget, the search latency must satisfy $\tau_s \leq \text{SLO}_{\text{search}} - W(b)$.

To avoid circular dependency (as $W(b)$ depends on τ_s), we approximate this term using a queuing factor ϵ , leading to:

$$\tau_s = \frac{\text{SLO}_{\text{search}}}{1 + \epsilon} \quad (3)$$

In our setting, we set $\epsilon = 1$, as it represents the worst case where the queuing delay equals one batch latency. This choice

is empirically supported from the CPU-only baseline, where ϵ ranged between 0.9 and 1.0.

Search iteration. The algorithm then performs a binary search over possible values of ρ using the modeled latency and hit rate behavior. For each candidate ρ , the reduced LLM throughput is estimated based on the corresponding decrease in KV cache capacity. Although this interpolation is coarse, it provides a conservative lower bound because the throughput–cache curve is generally convex. The INFERPARTITION function is subsequently invoked to compute the expected batch size, given by $B = \mu \cdot \tau_s$, where μ is the current throughput bound. Since batch size B must be an integer, two rounding strategies are considered:

- **Rounding up.** This implies longer latency and thus requires more cache coverage to meet τ_s . From the hybrid latency model (Eq. 1), we solve for η_1 and convert it to coverage ρ_1 via the HITRATE2COVERAGE function.

- **Rounding down.** This yields a smaller batch size (shorter latency), but may not meet the required throughput. To ensure throughput μ is met, we solve for η_2 using the adjusted latency bound B/μ from the throughput constraint.

At the end of the iteration, the smaller of ρ_1 and ρ_2 is selected, as it requires less GPU memory. This value is used to update the binary search interval.

Convergence. If the newly computed partitioning point ρ increases, the resulting drop in throughput leads to a smaller batch size in the next iteration, which in turn drives ρ back down. Conversely, if ρ shrinks, the throughput bound increases, allowing for more cache coverage. This feedback loop ensures convergence of the algorithm within a limited number of iterations. In practice, convergence takes less than one minute as shown in Figure 9.

4) *Index Splitter:* Once the partitioning point ρ is determined, it is passed to the final stage of index construction, which is the index splitter. The splitter first identifies the hot clusters based on the access profile and the target cache coverage ρ . These hot clusters are then sorted by size and distributed to GPU shards in a round-robin fashion to balance memory usage across sub-indexes.

Alongside the construction of each sub-index, the splitter generates a set of mapping tables. These tables encode the correspondence between original cluster IDs and their assigned shard as well as the remapped local cluster IDs, enabling efficient routing during query execution.

B. Distributed VectorLiteRAG Pipeline

The right side of Figure 7 illustrates the runtime architecture of VECTORLITERAG. At initialization, memory is allocated sequentially for the index and then for the LLM to prevent memory interference between the vector search and LLM engines. The two components operate through different processes and thus use separate GPU streams for concurrency.

Similar to other IVF-based indexes, the pipeline begins with coarse quantization to identify candidate clusters. However, from this point on, VECTORLITERAG introduces a

Algorithm 1 Latency Bounded Partitioning

Input: SLO_{search} , $MEM_{KVcache}$, μ_{LLM0}

Output: ρ

```

1:  $\tau_s \leftarrow \frac{SLO_{search}}{1+\epsilon}$ 
2:  $\rho_{low} \leftarrow 0$ ,  $\rho_{high} \leftarrow 1$ 
3: while  $\rho_{high} - \rho_{low} > \delta$  do
4:    $\rho_m \leftarrow \frac{\rho_{low} + \rho_{high}}{2}$ 
5:    $\mu_{LLM} \leftarrow \frac{MEM_{KVcache} - MEM_{Index}(\rho)}{MEM_{KVcache}} \mu_{LLM0}$ 
6:    $\rho \leftarrow \text{INFERPARTITION}(\tau_s, \mu_{LLM})$ 
7:   if  $\rho > \rho_m$  then
8:      $\rho_{low} \leftarrow \rho$ 
9:   else
10:     $\rho_{high} \leftarrow \rho_m$ 
11:   end if
12: end while
13: return  $\rho$ 
14:
15: function INFERPARTITION( $\tau_s, \mu$ )
16:    $B \leftarrow \lceil \tau_s \cdot \mu \rceil$ 
17:    $T_{search}^{CPU}(B), T_{LUT}^{CPU}(B) \leftarrow \text{PERFMODEL}(B)$ 
18:    $\eta_1 \leftarrow \frac{T_{search}^{CPU}(B) - \tau_s}{T_{LUT}^{CPU}(B)}$ 
19:    $\rho_1 \leftarrow \text{HITRATE2COVERAGE}(\eta_1, B)$ 
20:    $B \leftarrow \lfloor \tau_s \cdot \mu \rfloor$ 
21:    $T_{search}^{CPU}(B), T_{LUT}^{CPU}(B) \leftarrow \text{PERFMODEL}(B)$ 
22:    $\eta_2 \leftarrow \frac{T_{search}^{CPU}(B) - B/\mu}{T_{LUT}^{CPU}(B)}$ 
23:    $\rho_2 \leftarrow \text{HITRATE2COVERAGE}(\eta_2, B)$ 
24:   return  $\min(\rho_1, \rho_2)$ 
25: end function

```

customized retrieval pipeline tailored for hybrid CPU-GPU execution. We now describe each component in detail.

1) *Router:* To support efficient vector retrieval on a distributed multi-GPU system, VECTORLITERAG implements a custom routing mechanism rather than relying on Faiss’s built-in IndexIVFShards. The default implementation in Faiss is suboptimal in constrained environments for two main reasons. (1) IndexIVFShards partitions the index uniformly by vector or cluster ID, ignoring access frequency. While, convenient for implementation, it retains centroid metadata even for clusters that are not locally resident, causing unnecessary memory overhead, especially problematic when the number of clusters is large. (2) During search, each sub-index is instructed to probe the same number of clusters, even if many of them are not resident on that shard. Although certain probes are ultimately skipped at runtime, the batched execution of cluster scanning kernels still launches GPU thread blocks for them. These launches consume scheduling bandwidth and shared memory resources, regardless of whether the actual computation is needed. Since shared memory usage increases with n_{probe} , this results in inefficient kernel launches and exacerbates resource contention, especially in large-scale vector databases.

To address these issues, VECTORLITERAG uses the mapping tables generated during index splitting to route each query to the appropriate GPU shards and prune irrelevant probes, thereby accounting for the device-level variance. This substantially reduces the effective n_{probe} per shard, lowering both memory pressure and kernel scheduling overhead. At runtime, only GPU workers holding relevant clusters receive and execute the search request, while the remaining portion of the search is handled by the CPU. This hybrid execution minimizes contention and enables more efficient use of GPU memory and compute resources.

2) *Dynamic Dispatcher*: Because hit rates vary across queries, the effective n_{probe} differs even within a batch. As batch size increases, the minimum hit rate tends to decrease, increasing the search latency for the entire batch. To mitigate this issue, VECTORLITERAG employs a dynamic dispatcher that accelerates early query completion.

When search is initiated, a separate dispatcher thread is launched. Each GPU worker sets a completion flag once its assigned clusters are scanned. After all GPU flags are set, the dispatcher begins polling for queries that have completed their full search. To facilitate timely query promotion, a callback mechanism connects the CPU search loop and the dispatcher, as CPU processes clusters one-by-one, grouped by related queries. At the end of each iteration, the current scan count is compared with the expected n_{probe} for each query. When all assigned clusters for a query are scanned, the callback is invoked, and the query and its results are inserted into a thread-safe queue.

The dispatcher polls this queue at short intervals. Once a completed query is available, it merges the CPU and GPU results, re-ranks them to obtain the final top- k vectors, and forwards the result to the downstream document retriever. This proactive execution reduces head-of-line blocking within batches and improves end-to-end latency, particularly for high-hit-rate queries. It also enhances batching continuity by enabling smoother transitions between retrieval and generation stages, which already employs continuous batching schemes.

3) *Adaptive Runtime Index Update*: Our model is built upon the distributional characteristics of queries aggregated across batches. While correlations among queries may temporarily shift access patterns, they primarily reduce the number of statistically independent samples rather than altering the overall distributional trend. Nevertheless, temporal bias can arise in practice, and to mitigate potential performance degradation caused by such drift, VECTORLITERAG employs an adaptive re-profiling and update process.

VECTORLITERAG can swiftly react to shifts in query distribution without interrupting service. During runtime, the router monitors (1) average hit rates and (2) per-cluster access frequencies. For every few minutes or after a few thousand requests, it periodically resets the counters to detect distributional drift. When the average SLO attainment falls below a threshold and observed hit rates diverge from their expected values, an update cycle is triggered: re-profiling query access patterns, rerunning the latency-bounded partitioning algorithm,

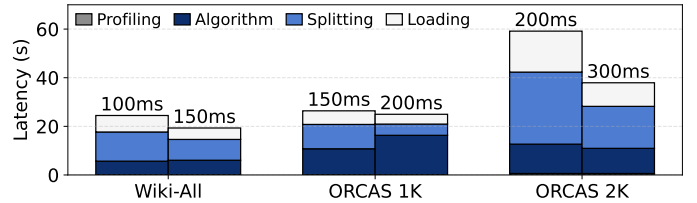


Fig. 9. Time consumed for re-building the GPU index shards using updated query access data. Numbers above the bars denote the search time SLO constraints applied for the system.

generating shards, and loading the updated indices onto GPUs.

All stages, from profiling to loading, complete in under a minute, allowing updates to run in the background. At the per-shard level, index generation and loading take less than ten seconds. The detailed timing breakdown for each stage is shown in Figure 9. While a GPU shard is being refreshed, the router temporarily redirects queries for those clusters to CPU paths, preserving the service continuity. Once the updated shard is loaded, routing automatically returns to the GPU.

Per-cluster updates are avoided because clusters are stored contiguously to enable high-bandwidth access. Since clusters vary in size, updating clusters individually would lead to memory fragmentation and inefficient data placement. Instead, VECTORLITERAG performs full-shard updates, as migration of each shard takes only a few seconds, providing robustness and simplicity.

According to our observations, profiling with only 0.5% of the queries from a separate training set successfully captured the distribution of 10M ORCAS queries. We therefore assume that a single index update can sustain stable service for roughly one hour under steady traffic, given the system throughput measured in our experiments.

V. METHODOLOGY

A. Experiment Setup

To evaluate VECTORLITERAG, we conduct experiments across various datasets, models, and hardware configurations. This section describes the datasets, models, evaluation metrics, and system setup.

Datasets and Models. We use two datasets: Wiki-All and ORCAS. We construct the IVF index following the configuration guidelines provided by the Faiss library. The Wiki-All [37] vector database contains 88M 768-dimensional vectors derived from Wikitext [28] and Cohere Wikipedia embeddings, yielding a compressed IVF index with a footprint of 18GB. We also construct two additional indexes from chunked Wikipedia documents [40] using the Stella [42] embedding model of dimensions 1024 and 2048, and queries from the Microsoft ORCAS dataset [5]. ORCAS consists of real Bing queries and preserves duplicates to reflect realistic query distributions. The ORCAS 1K and ORCAS 2K indexes occupy 40GB and 80GB of memory, respectively.

Our retrieval pipeline builds on Faiss v1.9.0 [6], [17], with internal extensions for flexible n_{probe} settings and dispatcher callbacks. The overall system, including the profiler and latency-aware scheduler, is implemented in Python.

For generation, we evaluate three models—Llama3-8B, Qwen3-32B, and Llama3-70B [9], [41]—served using vLLM v0.9.1 [18]. The retriever and LLM run as separate subprocesses, with the main process coordinating request generation and document fetching to integrate the full RAG pipeline.

To evaluate system performance, we sample queries from a dedicated test set that is disjoint from the profiling set. The request arrival process follows a Poisson distribution, a commonly adopted modeling choice in prior work [18], [31], [45]. For each query, the top-25 documents are retrieved, and a 1024-token input is constructed and passed to the LLM, which then generates a 256-token output, following the setup in [35]. The initial n_{probe} is set to 2048, which is sufficient to achieve an average retrieval quality of 0.91 Normalized Discounted Cumulative Gain (NDCG) [39] at 50.

SLO Settings. The SLOs for retrieval and generation stages were defined separately and then combined. For retrieval, since no standard criteria exist, we set the SLOs heuristically, relaxing them for larger databases (see Table I). For generation, the SLO was defined as the latency measured at the model’s throughput limit. These capacity values were also used in building our performance model.

TABLE I
SLO TARGET VALUES USED IN THE MAIN EVALUATION

Vector Index	SLO_{search}	LLM	SLO_{LLM}
Wiki-All	150ms	Llama3-8B	217ms
ORCAS 1K	200ms	Qwen3-32B	191ms
ORCAS 2K	300ms	Llama3-70B	311ms

System Configuration. We conduct our experiments on two types of nodes, each equipped with eight NVIDIA GPUs. The L40S node includes L40S GPUs with 48GB GDDR memory and dual Xeon 6426Y CPUs. The H100 node uses H100 GPUs with 80GB HBM and Xeon Platinum 8462Y CPUs. We use the L40S node for smaller models (Llama3-8B), while larger models requiring model parallelism (Qwen3-32B, Llama3-70B) are run on the H100 node for maximum throughput.

Baseline Configurations. We compare VECTORLITERAG against several key baselines. Since VECTORLITERAG builds on FAISS, we use vanilla FAISS-CPU IVF FastScan (CPU-Only), FAISS-GPU IVF on a dedicated GPU (DED-GPU), and a sharded FAISS-GPU IVF index distributed across all GPUs (ALL-GPU). To further demonstrate the strength of our approach, we also compare against HedraRAG [11] in section VI-D, which also uses a skew-aware caching strategy.

VI. EVALUATIONS

A. Performance Model and Hit Rate Estimator

Figure 10 evaluates the accuracy of VECTORLITERAG’s performance model. The right panel compares the predicted and actual minimum hit rates within each batch. As expected from order statistics, the minimum hit rate declines rapidly as batch size increases, and the rate of decline gradually flattens in the large-batch regime. Close alignment of two curves confirms that our Beta-distribution-based approximation reliably captures caching effectiveness.

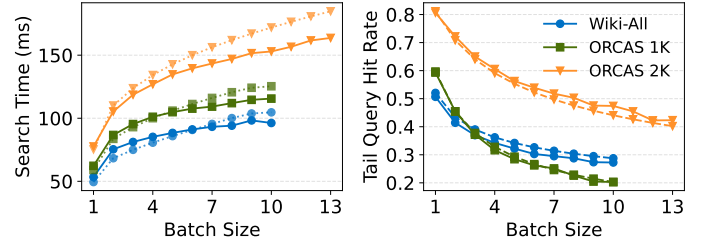


Fig. 10. Comparison of measured (solid line) vs estimated (dotted line) values from VECTORLITERAG’s performance model. **Left:** Search latency across batch sizes. **Right:** Tail hit rates within a batch.

The left panel compares the predicted latency of the hybrid index search with the measured latency. While the predictions generally follow the same trend, a noticeable offset exists between the two. This deviation mainly results from the dispatcher’s early-query handling, as discussed in Section VI-E1.

Precisely capturing the dispatcher’s impact would require evaluating full order statistics to model per-request completion times, which greatly increases complexity while providing only marginal benefit. Despite these approximations, the resulting configurations perform robustly in practice, as shown in the following sections.

B. SLO Attainment

Figure 11 presents SLO attainment curves across all nine combinations of vector databases and LLMs. In each subplot, the horizontal dashed line marks the 90th percentile latency target, and the vertical dashed line indicates the standalone LLM throughput. All experiments use on-demand dynamic batching, where retrieval requests are served immediately after the previous search completes, allowing throughput to scale with arrival rate through adaptive batch sizing.

Across all configurations, VECTORLITERAG sustains the extended SLO budget ($SLO_{\text{LLM}} + SLO_{\text{Search}}$, defined in Table I) over the widest input rate ranges among evaluated baselines. CPU-based fast scan can support relatively high request per second (RPS) rates, its limited per-request performance leads to consistent SLO violations even under light traffic. As arrival rate increases, batch sizes grow (up to 9–10 under >40 RPS), incurring high latency and poor tail response.

Dedicated GPU retrieval performs poorly with large models due to rigid model parallelism constraints. For instance, Llama3-70B requires a tensor parallelism degree of 4 for efficient execution. While it fits within 2 H100 GPUs, the achievable LLM throughput drops from 8 RPS to less than 2 RPS. In such settings, dedicating GPU(s) to retrieval results in resource oversubscription, harming overall system throughput.

For small vector databases and under light loads, ALL-GPU configurations can satisfy SLOs over wide traffic ranges. However, as the arrival rate approaches its reduced throughput, latency increases sharply. Although VECTORLITERAG is subject to this limitation as well, its optimized partitioning algorithm extends the SLO-attainable region nearly up to the standalone LLM throughput limit.

To better illustrate the dynamics of RAG systems, we present a detailed TTFT breakdown in Figure 12 for the

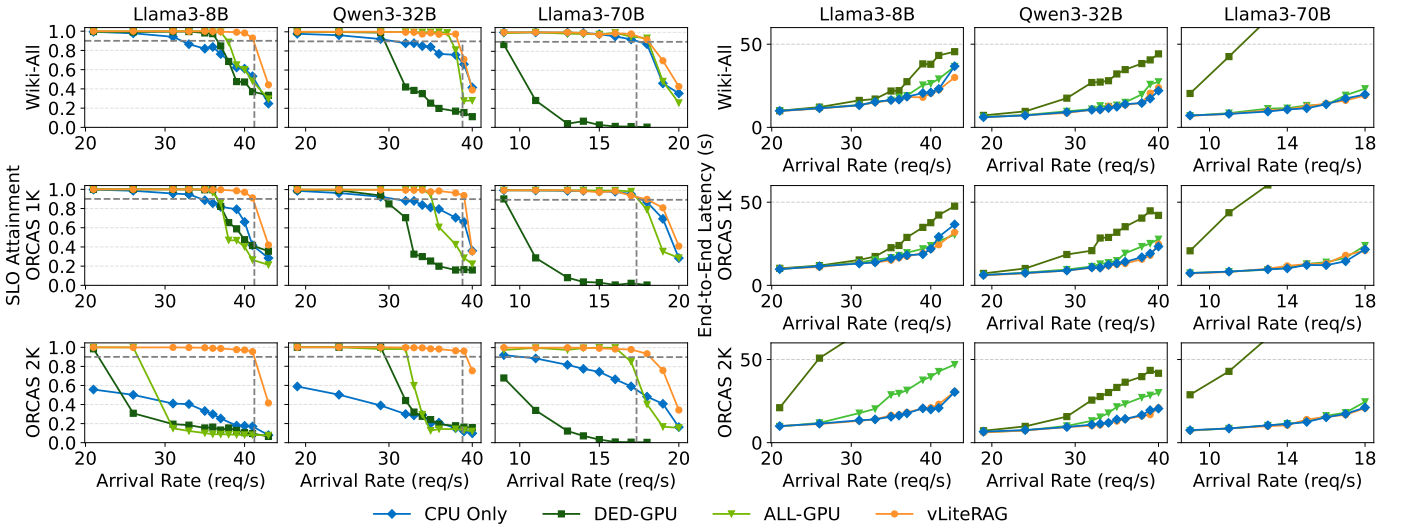


Fig. 11. **Left:** TTFT SLO attainment and **Right:** end-to-end latency of RAG pipeline under increasing arrival rates across different LLMs (columns) and datasets (rows). Our work (vLiteRAG) achieves higher SLO attainment across all regimes compared to baselines.

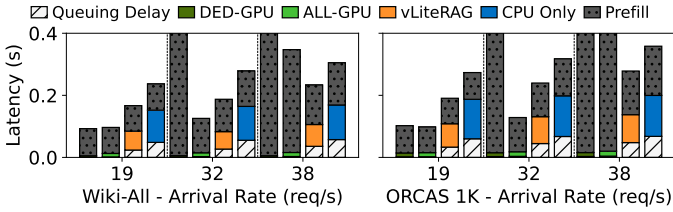


Fig. 12. TTFT breakdown for Wiki-All and ORCAS 1K indexes with Qwen3-32B. Each group shows results from four configurations. Bars are stacked to show the contribution of queuing delay, vector search latency (colored segments), and LLM prefill latency (grey)

Qwen3-32B model with Wiki-All and ORCAS 1K indices under varying input rates. As search latency increases, especially with CPU-based retrieval, queuing delays compound, further inflating TTFT. While both dedicated and ALL-GPU shared baselines perform well under low traffic, they exhibit latency spikes at higher rates due to resource contention. In contrast, VECTORLITERAG sustains stable latency by balancing throughput and latency, enabling finer control over resource allocation across the RAG stages.

C. End-to-End Latency

Since GPU resources are shared between retrieval and generation, interference with the decoding phase is inevitable. To assess the impact of such interference, we present the end-to-end latency results from the nine configurations discussed earlier, shown in Figure 11.

Retrieval contention is most severe for smaller models that can sustain higher loads, whereas large models saturate compute resources before retrieval pressure dominates. In the low-traffic regime, contention is minimal, except in DED-GPU, which reduces the number of GPUs available to the LLM. However, under high traffic and with large vector databases, contention becomes significant. This is evident in the more than $2\times$ increase in end-to-end latency observed in ALL-GPU baselines for ORCAS 2K with Llama3-8B and Qwen3-32B. Although Llama3-70B involves more intensive computation,

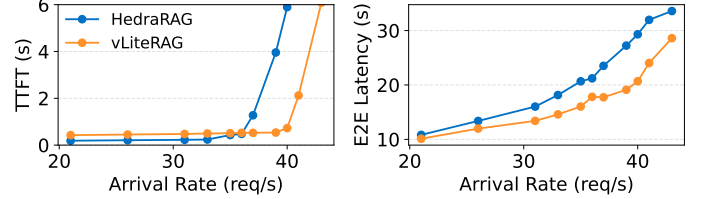


Fig. 13. Comparison with HedraRAG. HedraRAG exhibits lower TTFT at low request rates, but latency increases sharply once the system exceeds its throughput limit. VECTORLITERAG is configured with $SLO_{search} = 400\text{ms}$.

its low throughput ceiling causes TTFT to diverge before retrieval-induced interference becomes the dominant factor.

In contrast, VECTORLITERAG matches CPU-based retrieval in end-to-end latency. This demonstrates that its partitioning strategy and distributed execution pipeline effectively minimizes interference by carefully limiting GPU memory and usage of GPU threads for retrieval, thereby preserving LLM generation performance, while maintaining latency lower than SLO requirements.

D. Comparison with HedraRAG

We compare VECTORLITERAG with HedraRAG [11], which also exploits skewed cluster access patterns in RAG pipelines. While both systems adopt tiered caching strategies for vector indices, their partitioning principles and target objectives differ fundamentally.

HedraRAG selects GPU-resident clusters by identifying the maximum KV cache size that can sustain the throughput of the slower stage, either the LLM or the retriever. Although this approach is simple and throughput-aware, it does not account for latency constraints that are critical for real-time serving. In configurations where the LLM stage exhibits lower peak throughput than retrieval, as in 11, HedraRAG allocates the entire GPU memory to LLMs and performs vector search on the CPU. As noted in their paper, HedraRAG is most effective when retrieval becomes extremely heavy.

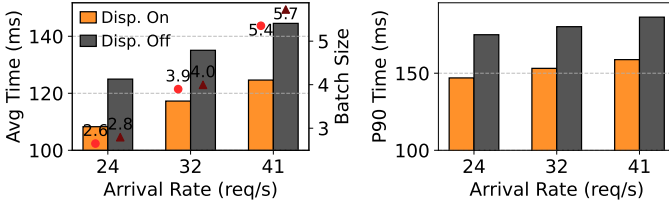


Fig. 14. **Left:** Average search latency and batch sizes. **Right:** P90 tail latency on ORCAS 2K index with dispatcher enabled and disabled.

To enable a fair comparison, we replicate the HedraRAG setting by building an IVF index with $\sqrt{N_{\text{vector}}}$ clusters and measuring retrieval throughput using batch sizes below 64. At $n_{\text{probe}} = 256$, CPU-only retrieval achieves 35 RPS at 0.94 NDCG@50; we increase n_{probe} to 6144 in our system to match this accuracy. Since HedraRAG does not support distributed retrieval, we apply their GPU caching scheme using `IndexIVFShard` without our optimized pipeline.

Figure 13 summarizes the results. HedraRAG places 73% of index clusters in GPU memory, whereas VECTORLITERAG identifies a partitioning point of 31.5% under a 400ms SLO. While HedraRAG achieves lower retrieval latency under low traffic, its operable range narrows as input rates increase. In contrast, VECTORLITERAG maintains latency near the target constraint across a wider traffic range and achieves lower overall end-to-end latency through its distributed pipeline.

The key distinction lies in how partitioning decisions are made. VECTORLITERAG allows operators to specify a target SLO and computes the largest GPU-resident index region that satisfies this constraint, whereas HedraRAG balances throughput between stages without explicit latency objectives, which can lead to suboptimal GPU allocation.

E. Ablation Studies

1) *Dynamic Dispatcher:* Figure 14 illustrates the effectiveness of the dynamic dispatcher in the distributed VECTORLITERAG pipeline. By polling the scanning loop and dispatching queries immediately upon completion, the dispatcher reduces search latency by up to 16%, improving both average and tail latency. This gain is achieved by overlapping the merging and re-ranking of early-completed queries with the ongoing scanning of slower queries, avoiding bulk merging at the end.

Figure 14 also reports average batch sizes under varying arrival rates. With adaptive batching, requests are grouped dynamically based on current pipeline load. Since vector search has higher throughput capacity than the LLM, it absorbs higher arrival rates by increasing batch size while maintaining stable service time. In contrast, fixed or capped batch sizes lead to request backlogs and performance degradation.

2) *Impact of LLM Input and Output Lengths:* Figure 15 illustrates latency sensitivity to varying input and output lengths for Llama3-8B and Llama3-70B. The red dashed line denotes the combined SLO target of vector search and LLM stages, corresponding to the 1024/256 setting in Table I. For consistency, SLO_{LLM} is fixed across configurations.

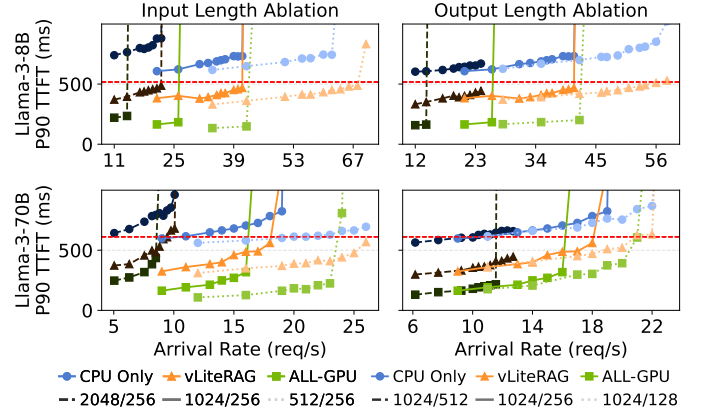


Fig. 15. **Left:** P90 TTFT across different input and **Right:** output lengths. Darker curves represent longer input/output sequences, while brighter curves correspond to shorter ones. Experiments were conducted using the ORCAS-2K index.

Longer inputs increase prefill cost, raising TTFT and shifting SLO violations to lower arrival rates as compute resources saturate. Similarly, longer outputs reduce the SLO-compliant range due to extended generation time and higher KV cache usage. Across both dimensions, VECTORLITERAG maintains serviceability over a wider range than the baselines, highlighting the robustness of its partitioning scheme.

3) *Sensitivity study on SLO_{search} :* To evaluate the robustness of our system under varying service constraints, we test VECTORLITERAG across multiple SLO_{search} targets. All plots in Figure16 use P95 TTFT as the primary metric, with P90 results additionally shown as dashed lines for VECTORLITERAG. Changing the quantile slightly expands or shrinks the SLO-compliant range; in our evaluation, the difference between P90 and P95 was at most 1 RPS.

TABLE II
SLO TARGETS AND CORRESPONDING INDEX SHARD SIZES.

SLO (ms)	Index (GB)	Param (GB)	KV Cache (GB)
100	3.80	30.59	33.24
150	2.95		34.09
200	2.47		34.57
250	2.21		34.83

Table II summarizes the target SLOs and their associated memory allocations. Under relaxed SLO constraints, the latency-bounded partitioning algorithm assigns a smaller fraction of the index to GPU shards, yielding latency behavior closer to the CPU-only baseline. As the SLO becomes stricter, the latency curve moves toward the all-GPU configuration. While tighter SLOs reduce available KV-cache space and modestly shrink the operable region, VECTORLITERAG still delivers a wider SLO-compliant throughput range than the baselines, highlighting the adaptability of its partitioning strategy and the effectiveness of its execution pipeline.

4) *Robustness to Hardware Capacity:* Finally, we evaluate how VECTORLITERAG adapts to different hardware capacities of the system. Following the provisioning policy commonly adopted by cloud providers, which allocates additional CPU cores as more GPUs are added, we test three

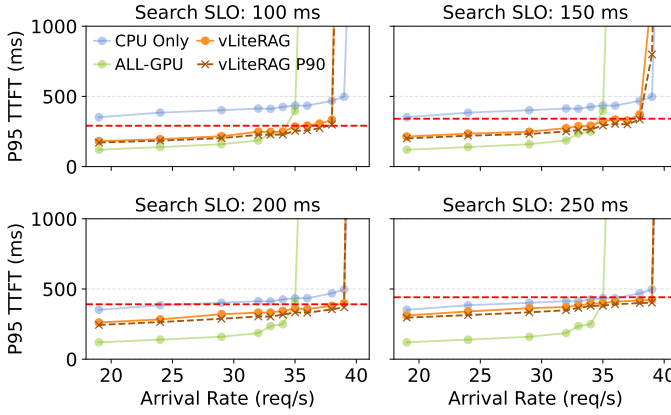


Fig. 16. P95 tail latency (and P90 for VECTORLITERAG) under different search-stage SLO constraints. Results are obtained using the Qwen3-32B model and the ORCAS 1K index.

configurations: 4 GPUs + 32 cores, 6 GPUs + 48 cores, and 8 GPUs + 64 cores. For each configuration, we re-profile the CPU-only search latency and apply the same latency-bounded partitioning algorithm. Aside from the number of compute devices, all experiments use identical model and index setups.

The results in Figure 17 show that VECTORLITERAG sustains the target SLO across all configurations while extending the SLO-compliant throughput roughly in proportion to the number of GPUs. While the reduced memory capacity in the GPU baseline causes decoding latency to grow rapidly with scale, VECTORLITERAG effectively contains this growth, keeping decoding latency comparable to CPU-only search cases. This demonstrates that VECTORLITERAG can be readily deployed across clusters of different sizes with minimal setup effort while maintaining consistent latency behavior.

VII. RELATED WORKS

RAG applications with iterative retrieval or multi-stage generation often exhibit semantic similarity across successive queries. Motivated by this observation, several optimization techniques have been proposed, including prefetching [23], speculative retrieval [44], and pipelined execution [15]. In contrast, our work builds upon application-agnostic, generic retrieval-generation pipelines without relying on semantic priors or intermediate signals. RagCache [16] improves throughput by managing KV cache reuse between tenants, focusing on scheduling and reuse optimizations on the LLM side. Hermes [35], on the other hand, scales via disaggregation by adding CPU nodes to offload vector search.

Efforts such as [12], [14], [21], [26], [32] propose specialized hardware or memory-centric architectures to accelerate RAG pipelines. While these approaches offer significant performance gains, they often rely on custom infrastructure, which may limit deployability in general-purpose environments. Among prior works, HedraRAG [11] also co-locates retrieval and generation on GPUs. Our work builds on this direction with an analytical model for latency and hit rate, enabling principled GPU memory partitioning under explicit SLOs. To our knowledge, VECTORLITERAG is the first

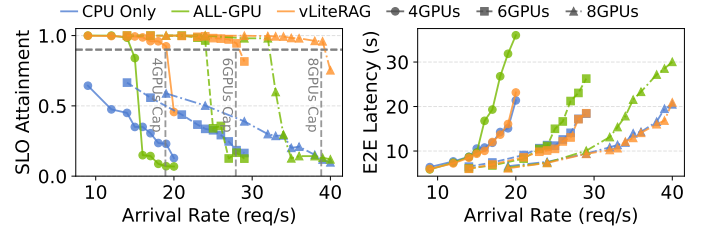


Fig. 17. **Left:** SLO attainment (the vertical dashed line denotes bare LLM capacity) and **Right:** end-to-end latency measured on 4-, 6-, and 8-GPU systems. Evaluated using the Qwen3-32B model and the ORCAS 2K index.

solution to provide fine-grained resource control for co-located RAG pipelines.

Future work may extend our approach to prefill-decode disaggregation frameworks [31], [45], where bandwidth-bound retrieval may run alongside compute-intensive prefill. This would require jointly modeling vector search and the throughput of both stages, but our framework offers a natural basis for such integration.

VIII. CONCLUSION

This paper presents VECTORLITERAG, a latency-aware orchestration framework for Retrieval-Augmented Generation (RAG) systems that explicitly manages the tight coupling between vector retrieval and LLM inference. We show that under skewed access patterns, variability in retrieval latency interacts with inference batching, causing tail effects amplification that cannot be mitigated by optimizing either stage in isolation.

VECTORLITERAG is driven by the insight that meeting strict RAG SLOs requires balancing batching behavior rather than maximizing instantaneous GPU utilization. By coordinating retrieval progress with inference scheduling, VECTORLITERAG suppresses tail cascades and sustains predictable end-to-end latency under bursty workloads. This enables SLO compliance across a substantially wider operating regime, supporting up to 1.5 $\times$ higher request rates than baseline RAG systems. Across extensive evaluation, we demonstrate that these benefits generalize across latency targets, hardware configurations, and LLM input/output lengths. VECTORLITERAG further exposes explicit control knobs that allow RAG operators to trade throughput for tail latency under constrained GPU memory budgets, making it practical for real-world deployment.

IX. ACKNOWLEDGMENT

This research was supported in part through cyber-infrastructure research resources and services provided by the Partnership for an Advanced Computing Environment (PACE) at the Georgia Institute of Technology, Atlanta, Georgia, USA. This work was partially supported by gifts from Google and AMD. The views and conclusions contained herein are those of the authors and should not be interpreted as representing the official policies or endorsements, either expressed or implied, of Georgia Tech.

A. Abstract

The artifact includes the complete source code of the coreVECTORLITERAG system, together with our modified FAISS library used for hybrid CPU–GPU vector search. To ensure reproducibility of both preprocessing and evaluation, we also provide a collection of shell scripts and Python utilities that automate the full experimental workflow, including dataset preparation, index construction, performance profiling, and end-to-end RAG pipeline evaluation. These scripts are designed to reproduce all major results reported in the paper.

All code and supporting materials are publicly available on GitHub <https://github.com/sitar-lab/VectorLiteRAG-AE> and Zenodo <https://zenodo.org/records/18195323>

B. Artifact Check-list

- **Program:** Modified FAISS library, vLLM
- **Compilation:** gcc-11.3, nvcc-12.1, cmake.
- **Models:** Llama-3 8B, Llama-3 70B, and Qwen-3 32B.
- **Datasets:** MS ORCAS and NVIDIA Wiki-All.
- **Run-time Environment:** RHEL 9 with Anaconda3.
- **Hardware:** Single node equipped with 8 NVIDIA L40S GPUs and 8 NVIDIA H100 GPUs.
- **Metrics:** SLO attainment, end-to-end latency, vector search hit rate estimation.
- **Output:** CSV logs and visualization plots.
- **Disk Space Required:** ~256 GB for evaluation; ~1.5 TB for preprocessing and index construction.
- **Workflow Preparation Time:** 40–50 hours.
- **Experiment Completion Time:** 10 hours.
- **Publicly Available?:** Yes.
- **Code Licenses?:** CC BY 4.0
- **Archived(DOI)?:** <https://zenodo.org/records/18195323>

C. Description

1) *How to access:* All source code and scripts are accessible via github repository.

2) *Hardware dependencies:* All experiments were conducted on a single L40S node or a single H100 node, each equipped with 8 GPUs. Because larger language models rely on tensor model parallelism, the H100 system is expected to provide NVLink connectivity to ensure reproducible performance. The L40S node was configured with a 32-core Intel CPU, and the H100 node with a 64-core Intel CPU. CPU core count is an important factor, as a substantial portion of the workload executes on the host processor.

3) *Software dependencies:* The evaluation environment was run on RHEL 9 (or a compatible Linux distribution) using Anaconda3. Successful compilation of the FAISS library depends on specific toolchain versions, including Python 3.10, GCC 11.3, and NVCC 12.1. Intel MKL is also required to support vectorized CPU operations.

4) *Datasets:* The Wikiall benchmark is directly downloadable. The ORCAS 1K and ORCAS 2K benchmarks require both the MS ORCAS dataset and the English Wikipedia dump, which are publicly accessible but require long preprocessing.

D. Installation and Testing

1) Installation:

```
# Create conda environment
cd VectorLiteRAG
conda create -n vlite -f ./scripts/env.yml
conda activate vlite
```

```
# Build faiss library
git submodule update --init --recursive
./scripts/build.sh
```

2) Preprocessing:

```
# Download a samll dataset for testing
./database/download.sh test
```

```
# Chunk documents and run embedding model
./database/encode.sh test
```

```
# Train index and construct base IVF
./scripts/train.sh test
```

3) Testing:

```
# Run a round of short cpu search based RAG
# Output csv files will appear under results/test
./scripts/test.sh
```

E. Experiment Workflow

We provide scripts for evaluation along with corresponding plotting utilities. The workflow is straightforward. After completing the preprocessing steps, execute the following commands from the project's root directory:

(1) Download the datasets: `./database/download.sh`

(2) Perform preprocessing and train the base indexes:

```
./database/encode.sh <dataset>
```

```
./scripts/train.sh <dataset>
```

(3) Run all experiments sequentially:

```
./scripts/runall.sh
```

For running experiments individually, please refer to the repository README.

(4) After all experiments are completed, generate the figures:

```
./scripts/plotall.sh
```

Individual plotting options are also documented in the README.

F. Evaluation and Expected Results

This artifact reproduces the primary experimental results presented in Figures 10–17 of the paper. During evaluation, latency logs are saved under `results/<datasets>`, and all vector indexes and their associated metadata are stored in `database/<dataset>`. After the evaluation completes, the provided plotting scripts generate the corresponding figures and place them in `figures` directory.

The reproduced results are expected to closely align with those reported in the paper, though minor variations may occur due to system-level factors. Individual evaluation runs for each data point are also supported and documented in the repository, enabling users to verify and assess any deviation.

Von

REFERENCES

- [1] M. Adnan, Y. E. Maboud, D. Mahajan, and P. J. Nair, "Accelerating recommendation system training by leveraging popular choices," *Proc. VLDB Endow.*, vol. 15, no. 1, p. 127–140, sep 2021.
- [2] M. Adnan, Y. E. Maboud, D. Mahajan, and P. J. Nair, "Ad-rec: Advanced feature interactions to address covariate-shifts in recommendation networks," 2023. [Online]. Available: <https://arxiv.org/abs/2308.14902>
- [3] M. Adnan, Y. E. Maboud, D. Mahajan, and P. J. Nair, "Heterogeneous acceleration pipeline for recommendation system training," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1063–1079.
- [4] F. André, A.-M. Kermarrec, and N. Le Scouarnec, "Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan," in *42nd International Conference on Very Large Data Bases*, vol. 9, no. 4, 2016, p. 12.
- [5] N. Craswell, D. Campos, B. Mitra, E. Yilmaz, and B. Billerbeck, "Orcas: 18 million clicked query-document pairs for analyzing search," *arXiv preprint arXiv:2006.05324*, 2020.
- [6] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *arXiv preprint arXiv:2401.08281*, 2024.
- [7] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on rag meeting llms: Towards retrieval-augmented large language models," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 6491–6501.
- [8] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [9] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billorec, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardaś, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasić, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenheide, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. Doudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Sweet, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelen, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A. L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabza, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dolla, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillion, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The llama 3 herd of models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [10] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "Retrieval augmented language model pre-training," in *International conference on machine learning*. PMLR, 2020, pp. 3929–3938.
- [11] Z. Hu, V. Murthy, Z. Pan, W. Li, X. Fang, Y. Ding, and Y. Wang, "Hedrarag: Coordinating llm generation and database retrieval in heterogeneous rag serving," *arXiv preprint arXiv:2507.09138*, 2025.
- [12] J. Jiang, H. Choi, H. Bae, S. Lee, M. Kwon, and M. Jung, "Cxl-anns: software-hardware collaborative memory disaggregation and computation for billion-scale approximate nearest neighbor search," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 585–600.
- [13] H. Jegou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE transactions on pattern analysis and machine intelligence*, vol. 33, no. 1, pp. 117–128, 2010.
- [14] W. Jiang, M. Zeller, R. Waleffe, T. Hoefler, and G. Alonso, "Chameleon: a heterogeneous and disaggregated accelerator system for retrieval-augmented language models," *arXiv preprint arXiv:2310.09949*, 2023.
- [15] W. Jiang, S. Zhang, B. Han, J. Wang, B. Wang, and T. Kraska, "Piperag: Fast retrieval-augmented generation via algorithm-system co-design," *arXiv preprint arXiv:2403.05676*, 2024.
- [16] C. Jin, Z. Zhang, X. Jiang, F. Liu, X. Liu, X. Liu, and X. Jin, "Ragcache: Efficient knowledge caching for retrieval-augmented generation," *arXiv preprint arXiv:2404.12457*, 2024.
- [17] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search

- with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [18] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.
 - [19] Y. Kwon and M. Rhu, “Training personalized recommendation systems from (gpu) scratch: Look forward not backwards,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 860–873.
 - [20] LangChain-Team, “Langchain: Context-aware reasoning framework,” <https://github.com/langchain-ai/langchain>, 2025, accessed: 2025-07-24.
 - [21] Y. Lee, H. Choi, S. Min, H. Lee, S. Beak, D. Jeong, J. W. Lee, and T. J. Ham, “Anna: Specialized architecture for approximate nearest neighbor search,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 169–183.
 - [22] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS ’20. Red Hook, NY, USA: Curran Associates Inc., 2020.
 - [23] C.-Y. Lin, K. Kamahori, Y. Liu, X. Shi, M. Kashyap, Y. Gu, R. Shao, Z. Ye, K. Zhu, S. Wang, A. Krishnamurthy, R. Kadekodi, L. Ceze, and B. Kasicki, “Telerag: Efficient retrieval-augmented generation inference with lookahead retrieval,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.20969>
 - [24] J. Liu, “Llamaindex,” https://github.com/jerryjliu/llama_index, Nov. 2022, released on November 1, 2022. [Online]. Available: https://github.com/jerryjliu/llama_index
 - [25] Y. E. Maboud, M. Adnan, D. Mahajan, and P. J. Nair, “Slipstream: Semantic-based training acceleration for recommendation models,” in *2025 Design, Automation & Test in Europe Conference (DATE)*, 2025, pp. 1–7.
 - [26] R. Mahapatra, H. Santhanam, C. Priebe, H. Xu, and H. Esmailzadeh, “In-storage acceleration of retrieval augmented generation as a service,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 450–466.
 - [27] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.
 - [28] S. Merity, C. Xiong, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.
 - [29] D. Mudigere, Y. Hao, J. Huang, Z. Jia, A. Tulloch, S. Sridharan, X. Liu, M. Ozdal, J. Nie, J. Park, L. Luo, J. A. Yang, L. Gao, D. Ivchenko, A. Basant, Y. Hu, J. Yang, E. K. Ardestani, X. Wang, R. Komuravelli, C.-H. Chu, S. Yilmaz, H. Li, J. Qian, Z. Feng, Y. Ma, J. Yang, E. Wen, H. Li, L. Yang, C. Sun, W. Zhao, D. Melts, K. Dhulipala, K. Kishore, T. Graf, A. Eisenman, K. K. Matam, A. Gangidi, G. J. Chen, M. Krishnan, A. Nayak, K. Nair, B. Muthiah, M. khorashadi, P. Bhattacharya, P. Lapukhov, M. Naumov, A. Mathews, L. Qiao, M. Smelyanskiy, B. Jia, and V. Rao, “Software-hardware co-design for fast and scalable training of deep learning recommendation models,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 993–1011. [Online]. Available: <https://doi.org/10.1145/3470496.3533727>
 - [30] A. Neelakantan, T. Xu, R. Puri, A. Radford, J. M. Han, J. Tworek, Q. Yuan, N. Tezak, J. W. Kim, C. Hallacy, J. Heidecke, P. Shyam, B. Power, T. E. Nekoul, G. Sastry, G. Krueger, D. Schnurr, F. P. Such, K. Hsu, M. Thompson, T. Khan, T. Sherbakov, J. Jang, P. Welinder, and L. Weng, “Text and code embeddings by contrastive pre-training,” 2022. [Online]. Available: <https://arxiv.org/abs/2201.10005>
 - [31] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative llm inference using phase splitting,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 118–132.
 - [32] D. Quinn, M. Nouri, N. Patel, J. Salihu, A. Salemi, S. Lee, H. Zamani, and M. Alian, “Accelerating retrieval-augmented generation,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 15–32.
 - [33] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlga, A. Shashua, K. Leyton-Brown, and Y. Shoham, “In-context retrieval-augmented language models,” *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1316–1331, 2023.
 - [34] N. Reimers, “Sentence embeddings using siamese bert-networks,” *arXiv preprint arXiv:1908.10084*, 2019.
 - [35] M. Shen, M. Umar, K. Maeng, G. E. Suh, and U. Gupta, “Hermes: Algorithm-system co-design for efficient retrieval-augmented generation at-scale,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 958–973.
 - [36] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, “Mpnnet: Masked and permuted pre-training for language understanding,” *Advances in neural information processing systems*, vol. 33, pp. 16 857–16 867, 2020.
 - [37] R. A. Team, “Wiki-all dataset,” https://docs.rapids.ai/api/cuvs/stable/cuvs_bench/wiki_all_dataset/, 2024, accessed: 2025-07-31.
 - [38] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu, K. Yu, Y. Yuan, Y. Zou, J. Long, Y. Cai, Z. Li, Z. Zhang, Y. Mo, J. Gu, R. Jiang, Y. Wei, and C. Xie, “Milvus: A purpose-built vector data management system,” in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 2614–2627. [Online]. Available: <https://doi.org/10.1145/3448016.3457550>
 - [39] Y. Wang, L. Wang, Y. Li, D. He, and T.-Y. Liu, “A theoretical analysis of ndcg type ranking measures,” in *Conference on learning theory*. PMLR, 2013, pp. 25–54.
 - [40] Wikimedia Foundation, “Wikipedia dumps,” <https://dumps.wikimedia.org/enwiki/latest/>, accessed: 2025-12-01.
 - [41] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, “Qwen3 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
 - [42] D. Zhang, J. Li, Z. Zeng, and F. Wang, “Jasper and stella: distillation of sota embedding models,” *arXiv preprint arXiv:2412.19048*, 2024.
 - [43] J. Zhang, Q. Liu, D. Lian, Z. Liu, L. Wu, and E. Chen, “Anisotropic additive quantization for fast inner product search,” in *Proceedings of the AAAI conference on Artificial Intelligence*, vol. 36, no. 4, 2022, pp. 4354–4362.
 - [44] Z. Zhang, A. Zhu, L. Yang, Y. Xu, L. Li, P. M. Phothilimthana, and Z. Jia, “Accelerating retrieval-augmented language model serving with speculation,” *arXiv preprint arXiv:2401.14021*, 2024.
 - [45] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.
 - [46] J. Zobel and A. Moffat, “Inverted files for text search engines,” *ACM computing surveys (CSUR)*, vol. 38, no. 2, pp. 6–es, 2006.